6. Introduction to OpenClaw Tools

Tools are the "capability interfaces" that OpenClaw exposes to agents, allowing models to complete real operations in a safer and more structured way, such as reading and writing files, running commands, managing sessions, accessing web pages, and UI automation. Compared with the legacy `openclaw-*` Skills, tools are typed and controllable, and configuration can precisely restrict "what it is allowed to do / what it is not allowed to do".

6. Introduction to OpenClaw Tools

1. Key Concepts
   - 1.1 The Relationship Between Tools and Skills
   - 1.2 More Tools Are Not Necessarily Better
   - 1.3 The Core of Tool Policy: profile + allow/deny
2. Quick Start
   - 2.1 Selecting a Tool Configuration Profile
   - 2.2 Precise Control with allow/deny (Recommended: Start with deny)
   - 2.3 Tighten Only for a Specific Provider/Model (byProvider)
3. Tool Groups (group:*)
4. High-Risk Tools: How to Configure Them Safely
   - 4.1 Command Execution Related (group:runtime)
   - 4.2 Tips for apply_patch
   - 4.3 Isolate Tools by Agent (Recommended Practice)
5. Recommended Configuration Templates (Ready to Use)
   - 5.1 Daily Dialogue (Safer)
   - 5.2 Studying/Writing Documents (Web Search Allowed, Running Commands Not Allowed)
   - 5.3 Development and Debugging (File Read/Write Allowed, but Running Commands Treated as "Optional")
6. Frequently Asked Questions
   - 6.1 allow Was Configured but Does Not Take Effect
   - 6.2 One Model Can Use Tools, but Another Cannot
   - 6.3 Which Profile Should I Choose?
7. Official Documentation

1. Key Concepts

1.1 The Relationship Between Tools and Skills

- Tools: system-level "capability interfaces", typed, configurable, and restrictable by provider/model
- Skills: more oriented toward "reusable workflows/plugin capabilities"; many legacy skills will be replaced or weakened by the tool system

If you want the agent to "do things", prioritize tools; if you want to encapsulate a process into a reusable capability, then consider Skills.

1.2 More Tools Are Not Necessarily Better

The more tools there are, the more the model can do, but the greater the risk. The best practice is: by default, enable only the tools you really need, and turn off high-risk capabilities (especially running commands) separately or enable them on demand.

1.3 The Core of Tool Policy: profile + allow/deny

The toolset is usually trimmed in this order:

1. `tools.profile` first provides a "basic allowed set"
2. If `tools.byProvider` is configured, the specified provider or provider/model is further tightened first (it can only narrow the toolset)
3. Finally, `tools.allow` / `tools.deny` are used to precisely tighten or loosen (with `deny` taking precedence)

2. Quick Start

OpenClaw's tool allow policy is written in `openclaw.json`. In general, we do not need to configure it manually

- `deny` takes precedence over `allow`
- The `*` wildcard is supported, and matching is case-insensitive

2.1 Selecting a Tool Configuration Profile

Common profiles (in order of "capability from least to most"):

- `minimal`: only `session_status`
- `messaging`: message-oriented capabilities + sessions capabilities
- `coding`: development/engineering-oriented capabilities (files, runtime, sessions, memory, images, etc.)
- `full`: no restrictions (the same as leaving it unset)

Example: enable the coding profile

```
{
  "tools": {
    "profile": "coding"
  }
}
```

2.2 Precise Control with allow/deny (Recommended: Start with deny)

Example: globally disable the `browser` tool

```
{
  "tools": {
    "deny": ["browser"]
  }
}
```

Example: on top of coding, disable all runtime tools (exec / bash / process)

```
{
  "tools": {
    "profile": "coding",
    "deny": ["group:runtime"]
  }
}
```

2.3 Tighten Only for a Specific Provider/Model (byProvider)

You can keep the global default unchanged and only further tighten the toolset for a specific provider or provider/model (it can only narrow, not expand).

Example: use coding globally, but minimal for Google Antigravity

```
{
  "tools": {
    "profile": "coding",
    "byProvider": {
      "google-antigravity": { "profile": "minimal" }
    }
  }
}
```

Example: further reduce tools for `openai/gpt-5.2`

```
{
  "tools": {
    "allow": ["group:fs", "group:runtime", "sessions_list"],
    "byProvider": {
      "openai/gpt-5.2": { "allow": ["group:fs", "sessions_list"] }
    }
  }
}
```

3. Tool Groups (group:*)

In `tools.allow` / `tools.deny`, you can use `group:*` to control a group of tools at once:

Group Name	Expands to
`group:runtime`	exec, bash, process
`group:fs`	read, write, edit, apply_patch
`group:sessions`	sessions_list, sessions_history, sessions_send, sessions_spawn, session_status
`group:memory`	memory_search, memory_get
`group:web`	web_search, web_fetch
`group:ui`	browser, canvas
`group:automation`	cron, gateway

Group Name	Expands to
`group:messaging`	message
`group:nodes`	nodes
`group:openclaw`	All built-in OpenClaw tools (excluding provider plugins)

Example: allow only file tools + browser

```
{
  "tools": {
    "allow": ["group:fs", "browser"]
  }
}
```

4. High-Risk Tools: How to Configure Them Safely

4.1 Command Execution Related (group:runtime)

`group:runtime` contains execution capabilities (for example, exec / bash / process). For such capabilities, the following is recommended:

- Disabled by default: first `deny: ["group:runtime"]`, then loosen it step by step when needed
- Open only to "specific agents": concentrate high-risk capabilities on dedicated agents
- Open only to "specific providers/models": grant more tools to stable/trusted models

4.2 Tips for apply_patch

`apply_patch` belongs to the file-editing tools; in some versions it is an experimental feature and may require an additional switch to enable (for example, via `tools.exec.applyPatch.enabled`, and only some models support it). If you find that a model clearly knows how to modify files but simply cannot call apply_patch, check the tool policy and the related switches first.

4.3 Isolate Tools by Agent (Recommended Practice)

You can set a global tool policy and then override it for a specific agent, so that "high-privilege capabilities" are concentrated on a small number of agents.

Example: use coding globally, but let the support agent use only messaging, and additionally allow Slack tools

```
{
  "tools": { "profile": "coding" },
  "agents": {
    "list": [
      {
        "id": "support",
        "tools": { "profile": "messaging", "allow": ["slack"] }
      }
    ]
  }
}
```

5. Recommended Configuration Templates (Ready to Use)

5.1 Daily Dialogue (Safer)

Goal: chat only + view the session status only; do not give it the ability to "mess with the system".

```
{
  "tools": {
    "profile": "minimal"
  }
}
```

5.2 Studying/Writing Documents (Web Search Allowed, Running Commands Not Allowed)

```
{
  "tools": {
    "profile": "coding",
    "deny": ["group:runtime"],
    "allow": ["group:web"]
  }
}
```

5.3 Development and Debugging (File Read/Write Allowed, but Running Commands Treated as "Optional")

```
{
  "tools": {
    "profile": "coding",
    "deny": ["group:runtime"]
  }
}
```

When you really need to execute commands, temporarily loosen `group:runtime`, and turn it back off after use.

6. Frequently Asked Questions

6.1 allow Was Configured but Does Not Take Effect

- Check whether `deny` simultaneously matches the same tool (deny takes precedence)
- Check whether the tool name is spelled correctly (matching is case-insensitive, but the name must exist)
- If `tools.allow` contains only unknown/not-loaded plugin tools, the system may ignore the allow list to ensure that core tools remain available

6.2 One Model Can Use Tools, but Another Cannot

- You may have set `tools.byProvider` (provider or provider/model) to impose additional restrictions on that model
- Some provider endpoints have inconsistent tool support; it is recommended to use a smaller toolset for unstable endpoints

6.3 Which Profile Should I Choose?

- Just want to chat: `minimal`
- Daily messaging + session management: `messaging`
- File/engineering tasks: `coding`
- You clearly know what you are doing and can bear the risk, and you want the complete, powerful OpenClaw: `full`

7. Official Documentation

- OpenClaw tool documentation: https://docs.openclaw.ai/zh-CN/tools